

CHAPITRE IV

Réduction de la consommation énergétique des systèmes embarqués

I. Principe générales

I. Intérêt et composants impactés:

Pourquoi réduire la consommation?

Rappel:

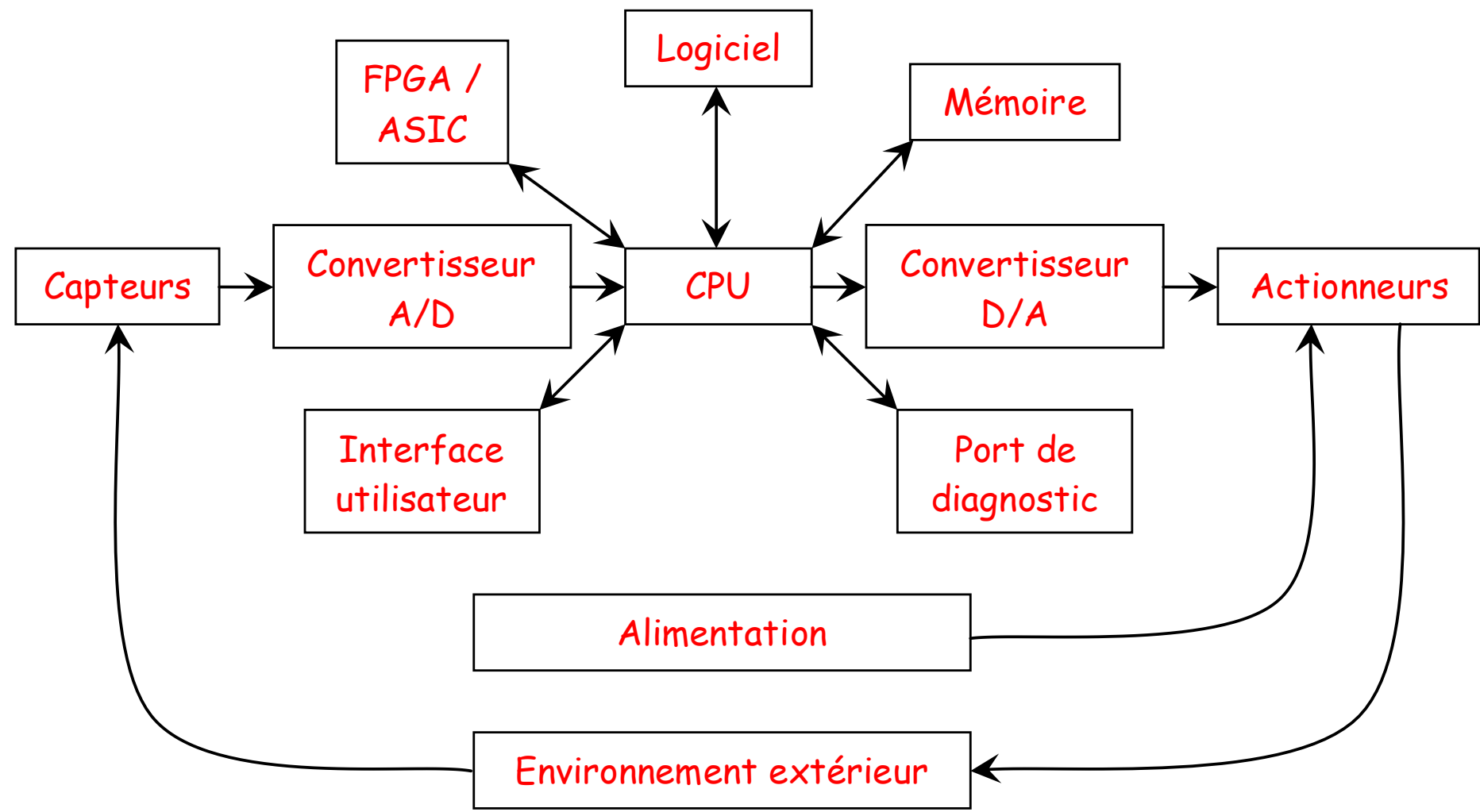
Un système embarqué (embedded system) est un système mixte contenant du matériel et du logiciel et qui est complètement intégré dans l'environnement qu'il contrôle.

Le système étant intégré dans son environnement, il se doit d'être autonome. Réduire la consommation énergétique globale du système revient à prolongé directement sa durée de vie.

I. Principe générales

I. Intérêt et composants impactés:

Sur quel composant agir?



I. Principe générales

I. Intérêt et composants impactés:

- Actionneur: Principal consommateur d'énergie.
- Capteur: dépendant de la technologie, peut au contraire produire de l'énergie plutôt que d'en consommer!
- Microcontrôleur: Peut consommateur mais souvent toujours actif dans le système
- IHM: peut être plus ou moins consommatrice (beaucoup dans le cas d'écran).

I. Principe générales

I. Intérêt et composants impactés:

Quand agir?

- En fonctionnement: Par définition le système embarqué doit produire des sorties juste dans un temps impartis. La réduction de la consommation ne doit pas concurrencer cette contrainte.
- Entre deux phases de fonctionnement: Le temps d'attente entre deux phases est propice à la réduction de consommation.

I. Principe générales

I. Intérêt et composants impactés:

Cas des Actionneurs et Capteurs:

La réduction de la consommation lors de ses deux catégories se fait majoritairement lors de la phase de conception du système. Un soin tout particulier aux technologies utilisées; focalisé sur le rendement pour l'actionneur et sur la prise de mesure avec participation de l'environnement extérieur comme source d'alimentation (ou simple contact sec) pour le capteur; permet la réduction la plus effective.

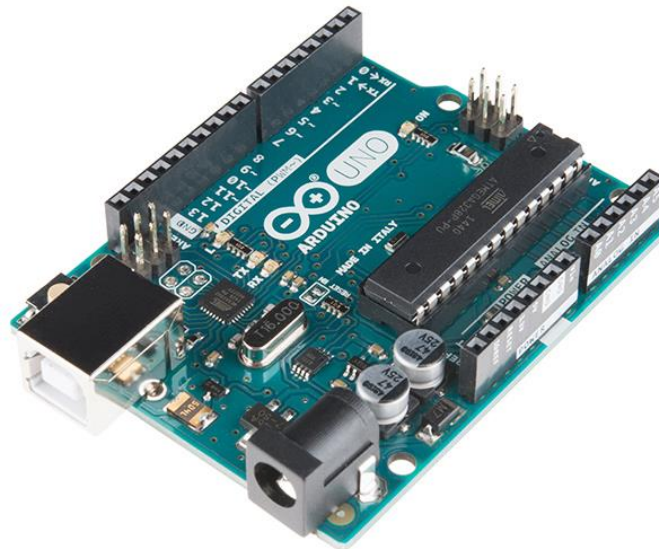
De plus, le microcontrôleur peut désactiver l'alimentation de ses parties (par pilotage de relai, transistor) lorsqu'elle ne sont pas utilisées, maximisant la réduction.

II. Réduction de la consommation des architectures à microprocesseur

I. Présentation de l'exemple:

Pour la partie microcontrôleur, la encore la phase de conception joue un rôle crucial. Pour s'en rendre compte, prenons un système non optimisé et essayons de réduire sa consommation dans chacune des fonctionnalités clés que l'on retrouve dans la plupart des systèmes embarqués:

Notre exemple porte sur une carte Arduino Uno



II. Réduction de la consommation des architectures à microprocesseur

I. Présentation de l'exemple:

carte Arduino Uno:

Microcontrôleur Atmega328P.

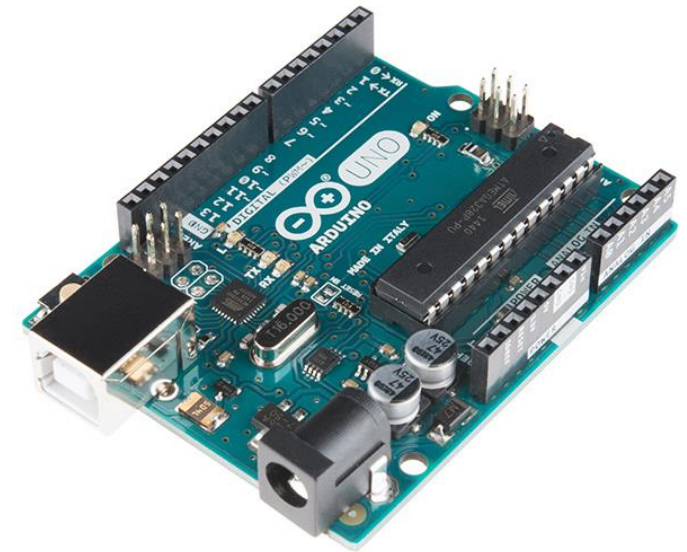
Régulation de la tension d'entrée (12V vers 5V et 3,3V)

Pinout de chacune des broches du microcontrôleur

Leds d'états et bouton MCLR

Interface USB (boot loader intégré
dans le microcontrôleur)

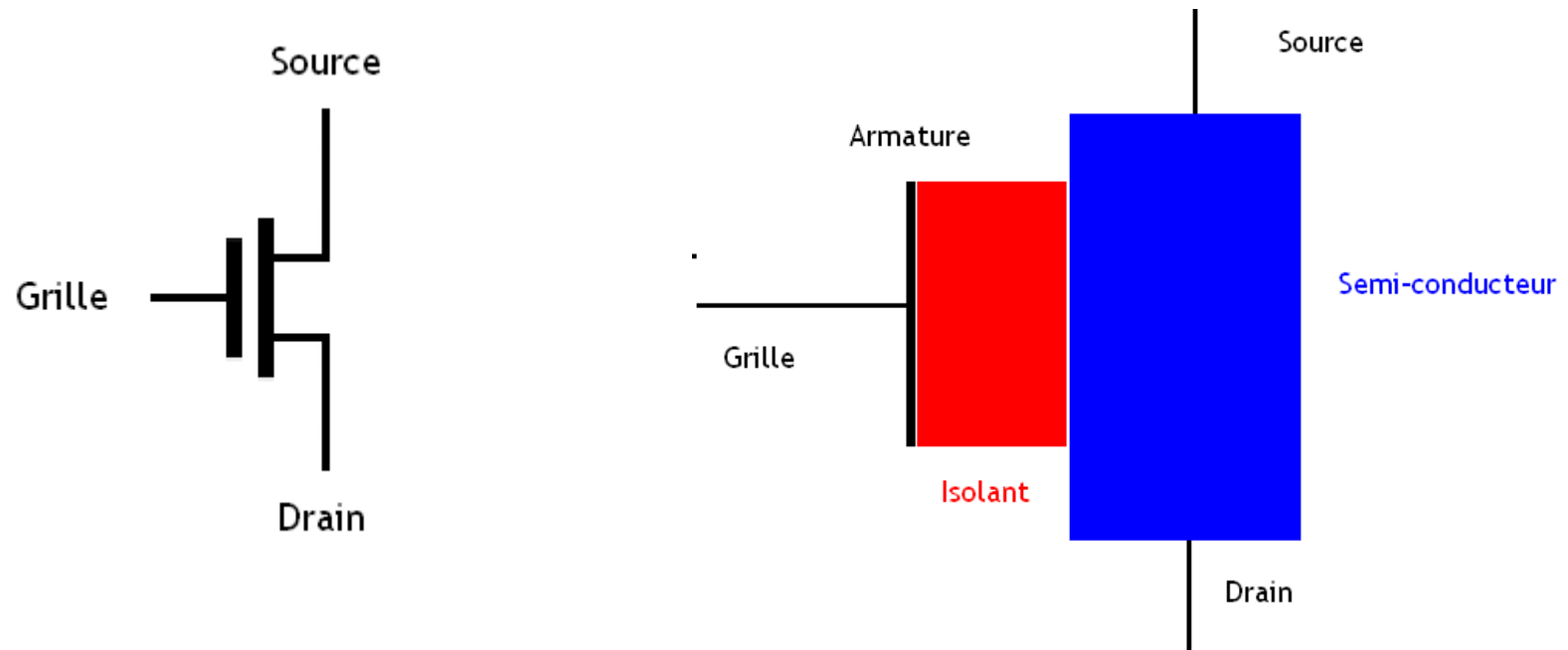
Quartz externe



II. Réduction de la consommation des architectures à microprocesseur

II. Réduction de la fréquence de fonctionnement du processeur:

Les processeurs sont constitués d'un ensemble de transistor de type CMOS:



II. Réduction de la consommation des architectures à microprocesseur

II. Réduction de la fréquence de fonctionnement du processeur:

Principe de fonctionnement (PMOS):

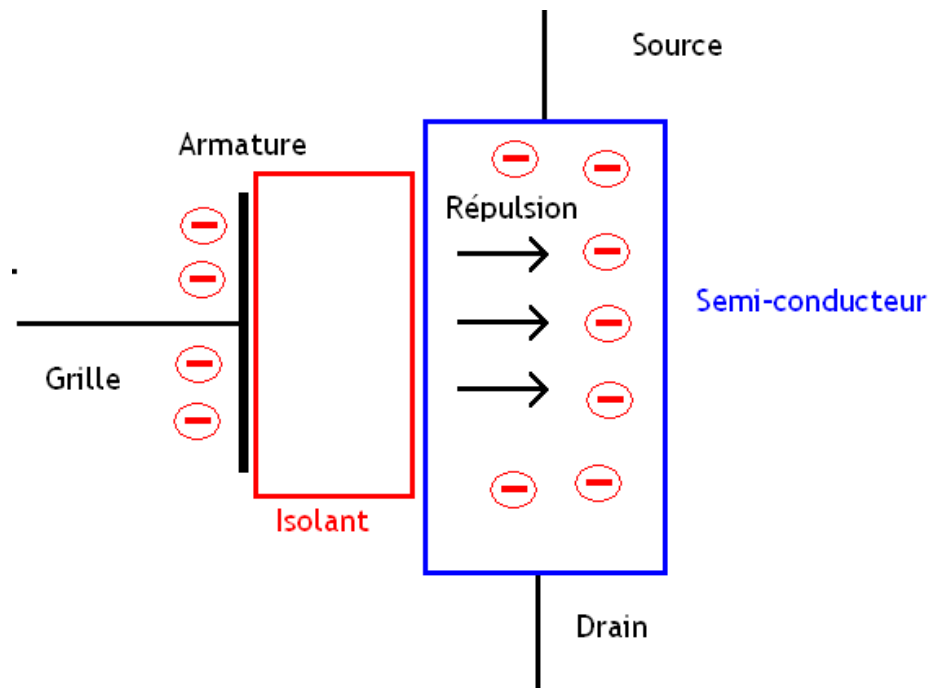
Premier cas : Aucune tension n'est appliquée sur la grille, la plaque de métal reliée à la grille est vide d'électrons. Si on place une tension entre la source et le drain, un courant électrique va passer et des électrons vont circuler de la source vers le drain : un courant va s'établir.

Maintenant, si on place une tension sur la grille, des électrons vont s'accumuler dans la plaque de métal de la grille jusqu'à un certain point. Au bout d'un certain temps, la grille sera remplie. Ces électrons chargés négativement vont donc repousser tous les autres électrons qui circulent entre la source et le drain, ce qui va les gêner et faire circuler ceux-ci plus difficilement : le courant va diminuer.

II. Réduction de la consommation des architectures à microprocesseur

II. Réduction de la fréquence de fonctionnement du processeur:

Principe de fonctionnement (PMOS):

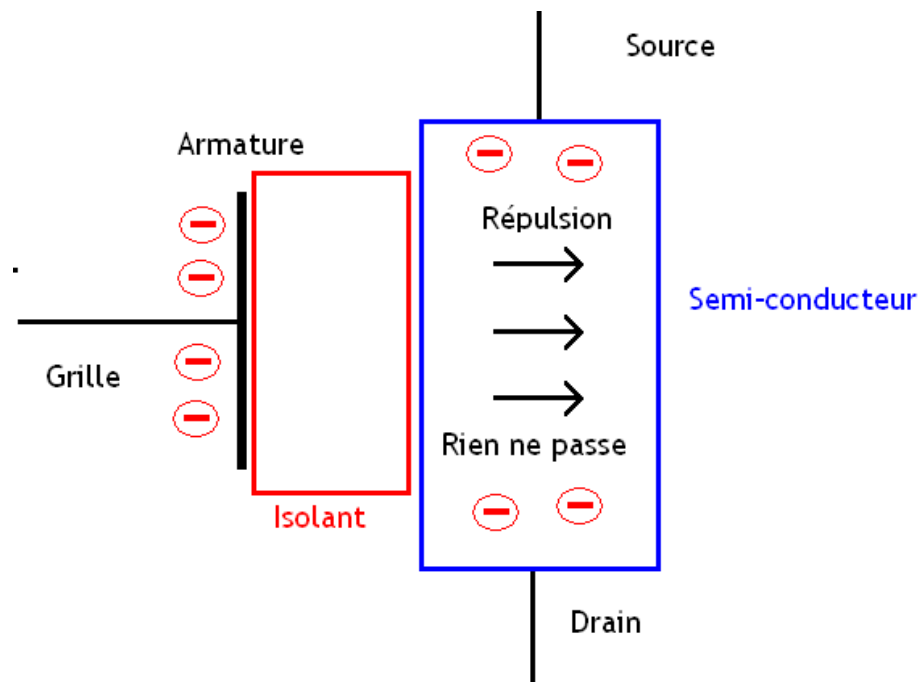


II. Réduction de la consommation des architectures à microprocesseur

II. Réduction de la fréquence de fonctionnement du processeur:

Principe de fonctionnement (PMOS):

À partir d'une certaine tension, les électrons stockés dans la grille vont tellement repousser les électrons de la source que ceux-ci ne traverseront plus du tout la liaison entre la source et le drain : le courant ne passe plus.



II. Réduction de la consommation des architectures à microprocesseur

II. Réduction de la fréquence de fonctionnement du processeur:

2 type de consommation peuvent être distinguées:

La consommation statique: Lorsque le transistor est passant, un courant circule de la source vers le drain.

La consommation Dynamique: A chaque changement d'état, le transistor doit accumuler et faire fuir des électrons sur la grille, il y a échauffement. Si l'on regarde la figure précédente: on remarque 2 parties conductrices (Grille et semiconducteur) séparé par un isolant. Le transistor peut donc être assimilé à un condensateur: Pour le rendre passant, il faut donc charger de l'énergie. On peut donc utilisé la formule suivante:

$$E = \frac{1}{2}CU^2$$

Avec E L'énergie accumulé, C la capacité et U la tension appliquée sur l'armature. Donc à chaque changement d'état, une énergie proportionnelle à U est consommée.

II. Réduction de la consommation des architectures à microprocesseur

II. Réduction de la fréquence de fonctionnement du processeur:

Lors de son fonctionnement, tous les transistors ne changent pas d'état à chaque coup d'horloge. Néanmoins posons l'hypothèse suivante:

nos transistors n'arrêtent pas de changer d'état et sont exploités au mieux!
Donc, pour un processeur de fréquence f , nos transistors vont changer d'état f fois par seconde.

Pour un transistor:

$$E = \frac{1}{2} f C U^2$$

Pour un processeur à n transistor:

$$E = \frac{1}{2} f n C U^2$$

II. Réduction de la consommation des architectures à microprocesseur

II. Réduction de la fréquence de fonctionnement du processeur:

Plus un processeur a de transistors, plus celui-ci consommera et chauffera. On peut aussi en déduire qu'un processeur consommera d'autant plus que sa fréquence et/ou sa tension d'alimentation sont élevées.

On voit donc plusieurs solutions simples pour diminuer la consommation énergétique d'un processeur :

- diminuer le nombre de transistors ;
- diminuer la capacité résiduelle de ces transistors ;
- diminuer la fréquence ;
- diminuer la tension d'alimentation du processeur.

II. Réduction de la consommation des architectures à microprocesseur

II. Réduction de la fréquence de fonctionnement du processeur:

Remarque:

les fabricants de CPU sont pleins d'astuces. Ils ont eu l'idée de faire varier la tension et la fréquence en fonction de ce que l'on demande au processeur. Rien ne sert d'avoir un processeur qui tourne à 200 gigahertz pendant que l'on regarde ses mails. Par contre, avoir un processeur à cette fréquence peut être utile lorsque l'on joue à un jeu vidéo dernier cri qui a besoin d'une quantité énorme de ressources. Dans ce cas, pourquoi ne pas adapter la fréquence suivant l'utilisation qui est faite du processeur ? C'est l'idée qui est derrière le Dynamic Frequency Scaling, aussi appelé DFS. Pour cela, il faut inclure dans les circuits du processeur une petite unité dédiée à la gestion de l'énergie. Cette unité estimera en permanence l'utilisation et la charge de travail imposée au processeur et en déduira s'il faut réduire la fréquence du processeur (et de combien) ou la laisser à son niveau maximal. Cette unité utilise différents algorithmes câblés sous forme de circuits qui analysent et adaptent la fréquence.

II. Réduction de la consommation des architectures à microprocesseur

II. Réduction de la fréquence de fonctionnement du processeur:

Implémentation:

Tout comme dans le STM32F407 (voir TP1), La fréquence d'un processeur est un multiple d'une fréquence de base (basé sur un quartz) : on obtient la fréquence du processeur en multipliant celle-ci par un coefficient multiplicateur. Pour effectuer cette multiplication de fréquence, l'horloge va passer par un circuit avant d'être distribuée à travers tout le processeur. Ce circuit est ce qu'on appelle une Phase Locked Loop, ou PLL. Le Dynamic Frequency Scaling consiste simplement à utiliser une PLL dont le coefficient multiplicateur est programmable, et de la commander suivant les besoins.

II. Réduction de la consommation des architectures à microprocesseur

II. Réduction de la fréquence de fonctionnement du processeur:

Efficacité:

Cette technologie est en soi assez peu efficace. Et cela pour plusieurs raisons : la consommation énergétique varie proportionnellement à la fréquence, ce qui a moins d'impact qu'une baisse de la tension d'alimentation. De plus, cela n'aura d'impact que sur les circuits qui changent d'état, à savoir les circuits qui font des calculs ou ceux qui sont reliés à l'horloge pour qu'ils se synchronisent avec elle. Diminuer la fréquence lorsque l'on a besoin de calculs serait inutile : on observerait une baisse des performances assez flagrante et donc incompatible avec un système avec des contraintes temps réelles; diminuer la fréquence quand peu de circuits changent d'état est peu efficace.

II. Réduction de la consommation des architectures à microprocesseur

II. Réduction de la fréquence de fonctionnement du processeur:

Application à l'Atmega328P

Utilisons le sketch suivant (A):

```
void setup () {}  
void loop () {}
```

L'équivalent d'un main vide sous Microchip. Voici la consommation relevée avec ce sketch

PLL Value	CPU Speed	Current
0xFF	16 MHz	15.15 mA
0xE2	8 MHz	11.05 mA
0x7F	2 MHz	7.21 mA
0x62	1 MHz	6.77 mA
0xE3	128 KHz	6.00 mA

La consommation est divisée par 2 en divisant la fréquence par 16.

Remarque: les courants sont obtenue avec un pcb « barebones » et non la carte arduino UNO. (voir slides suivantes)

II. Réduction de la consommation des architectures à microprocesseur

III. Réduction de l'alimentation du processeur:

Application à l'Atmega328P

Si l'on fixe maintenant l'horloge interne à 8Mhz et que l'on fait varier la tension d'alimentation (sans utiliser de régulateur):

- 5.0V : 11.67 mA
- 4.5V : 7.74 mA
- 4.0V : 5.60 mA
- 3.5V : 4.10 mA
- 3.3V : 3.70 mA
- 3.0V : 3.30 mA
- 2.8V : 3.00 mA
- 2.5V : 2.70 mA
- 2.4V : 2.50 mA
- 2.3V : 2.40 mA
- 2.2V : 2.30 mA
- 2.1V : 2.10 mA

La consommation décroît drastiquement.

II. Réduction de la consommation des architectures à microprocesseur

III. Réduction de l'alimentation du processeur:

Application à l'Atmega328P

Remarque: avec la fréquence réglée à 128Khz:

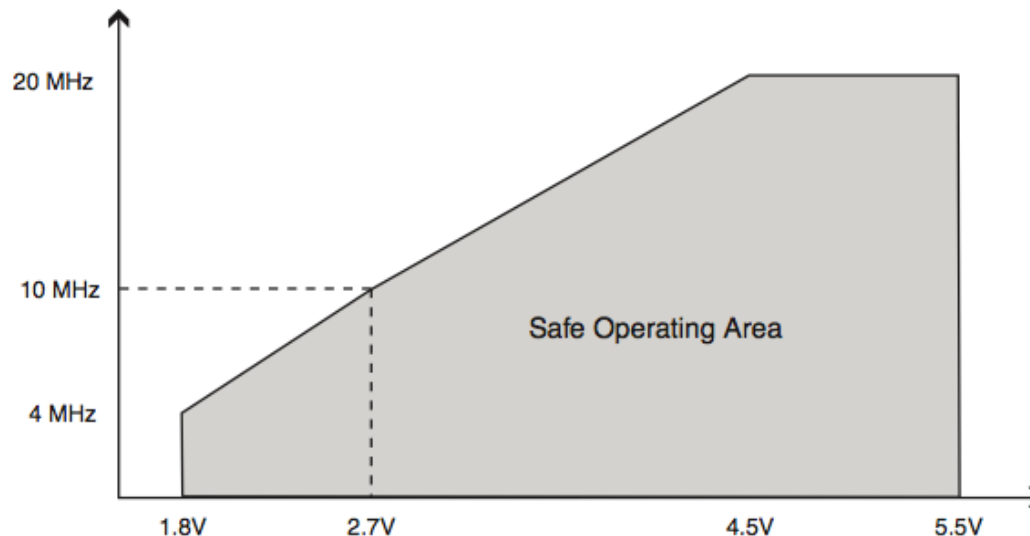
- 2,V: 0,19 mA
- 1,8V: 0,14 mA

II. Réduction de la consommation des architectures à microprocesseur

III. Réduction de l'alimentation du processeur:

Attention à respecter les limites imposé par le constructeur:

Figure 28-1. Maximum Frequency vs. V_{CC}



On peut en déduire:

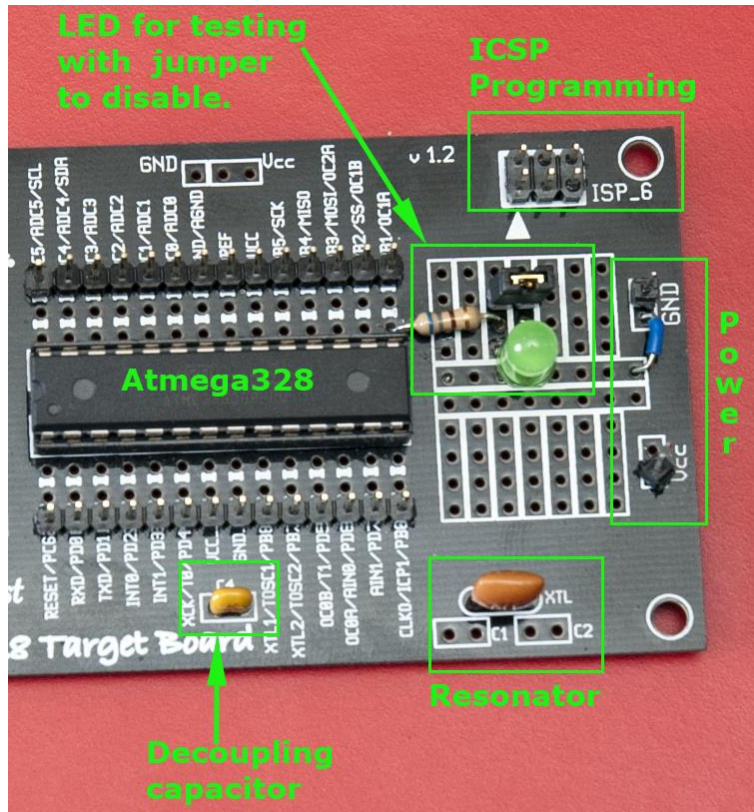
- De 1.8V to 2.7V: $F_{max} = 4 + ((V - 1.8) / 0.15)$
- De 2.7V to 4.5V: $F_{max} = 10 + ((V - 2.7) / 0.18)$

II. Réduction de la consommation des architectures à microprocesseur

III. Réduction de la connectique de la carte:

La carte Arduino Uno adopte une approche maximaliste pour réaliser tout type de Projet (Accès à tous les PIN, protocole de communication avec Drivers Présent, LEDs de test...)

Dans une approche minimaliste:



Résultat de consommation du Sketch A:

Carte Dev Uno: 50mA.

Carte Barebone: 15,15mA.

II. Réduction de la consommation des architectures à microprocesseur

III. Réduction de la connectique de la carte:

Remarques:

Si il doit être utilisé, le choix d'un bon régulateur de tension est cruciale:
Perte de joule très importante. ex: Le LM7805 consomme en permanence 5mA.

Une Led consomme en moyenne 10mA.

II. Réduction de la consommation des architectures à microprocesseur

IV. Optimisation logicielle:

Un logiciel pour système embarqué effectue périodiquement un ensemble de fonctionnalité. Entre chaque période, le processeur peut être endormie:

Soit le sketch suivant (B):

```
#include <avr/sleep.h>
void setup ()
{
  set_sleep_mode (SLEEP_MODE_PWR_DOWN);
  sleep_enable();
  sleep_cpu ();
} // end of setup void loop () { }
```

Le CPU est endormie. La consommation (barebone) est de 15,15mA pour le sketch A contre 360 uA pour le sketch B!

II. Réduction de la consommation des architectures à microprocesseur

IV. Optimisation logicielle:

Consommation en fonction des modes d'endormissement de l'Atmega:

- SLEEP_MODE_IDLE: **15 mA**
- SLEEP_MODE_ADC: **6.5 mA**
- SLEEP_MODE_PWR_SAVE: **1.62 mA**
- SLEEP_MODE_EXT_STANDBY: **1.62 mA**
- SLEEP_MODE_STANDBY : **0.84 mA**
- SLEEP_MODE_PWR_DOWN : **0.36 mA**

Le mode PWR_DOWN ne laisse que le Timer 2 tourner (alimenter par un quartz externe).

Dépendant de votre méthode de réveil et des fonctionnalités minimale nécessaire en mode endormie, vous ne serez pas toujours capable d'atteindre 360uA.

II. Réduction de la consommation des architectures à microprocesseur

IV. Optimisation logicielle:

Remarque: dépendant de l'état des E/S, la consommation endormie peut varier:

- **All pins as outputs, and LOW: 0.35 mA**
- **All pins as outputs, and HIGH: 1.86 mA.**
- **All pins as inputs, and LOW internal pull-ups disabled): 0.35 mA**
- **All pins as inputs, and HIGH internal pull-ups enabled): 1.25 mA.**

II. Réduction de la consommation des architectures à microprocesseur

IV. Optimisation logicielle:

L'endormissement concerne le processeur ainsi que quelques périphériques (Timer), d'autre pistes permette d'optimiser la réduction:

Les périphériques utilisant une tension de référence générée:

- Le Brown-out: détection de la tension basse. Economie de 25 μA
- L'ADC: comparaison du signal d'entré avec une référence: Economie de 335 μA !

On peut ainsi passer de 360 μA à 0,355 μA !

III. Espérance de vie du système

L'autodécharge est un phénomène qui touche toutes les technologies de batteries:

Type	Capacity mAh	Discharge %/month	Self discharge (µA)
CR1212	18	1	0.250
CR1620	68	1	0.950
CR2032	210	1	3
NiCD AAA	350	20	98
NiMH AAA	900	30	375
NiCd AA	1000	20	270
Alkaline AAA	1250	2	270
NiMH AA	2400	30	1000
Alkaline AA	2890	2	80
Li-Ion	4400	10	600

Remarque: Ces valeurs sont des approximations, l'autodécharge n'est pas linéaire en fonction de l'état de charge.

III. Espérance de vie du système

Concernant les batteries, les constructeurs donnent généralement des informations en taux de décharge par mois (en pourcentage de la capacité).

On en déduit la formule:

$$A_{\text{équivalent}} = \frac{\% \text{décharge par mois}}{100} * \frac{C}{24 * 30}$$

Avec C en mAh

III. Espérance de vie du système

Exercice:

En considérant que la tension des piles utilisées est toujours constante quelques soit la capacité restante, calculer combien de temps le système durera dans le cas de:

- Utilisation du sketch A avec la carte de développement à une fréquence de 16Mhz en utilisant des piles alcaline AAA (1,5V: Prendre la quantité minimale pour faire fonctionner le système)
- Même question avec des piles NIMH AA (1,5V)
- Utilisation du sketch B, ADC et Brown-out désactivé, fréquence de 16Mhz en utilisant des piles alcaline AAA
- Même question avec des piles NIMH AA (1,5V)